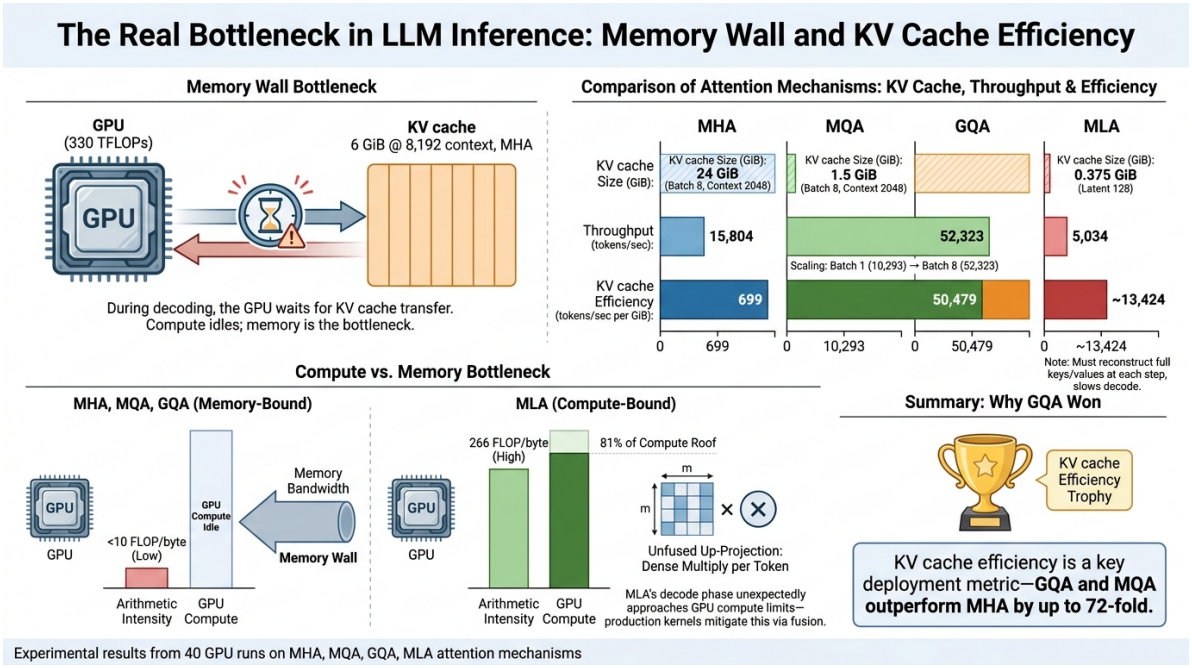
Attention Mechanism Stress Tester: Learning Notes

Manoj Chandrashekar Rao



1 Project Goal

The goal of this project is to understand attention mechanisms from an inference engineering perspective rather than from a training-only perspective. The main question is: how do MHA, MQA, GQA, and a simplified MLA-style latent KV cache change latency, throughput, memory use, and deployment tradeoffs?

The project is intentionally scoped around a reproducible benchmark harness, hardware-aware measurements, and a decision framework for deployment settings.

2 Step 1: GPU Roofline Setup

2.1 What I Did

I started by setting up the project on RunPod and generating a roofline plot for the GPU that will run the attention benchmarks. The machine used an NVIDIA GeForce RTX 4090. The roofline script detected GPU information through `torch` and `nvidia-smi`, then matched the GPU name against a small local hardware-spec catalog.

The detected benchmark-relevant hardware information was:

Property	Value
GPU	NVIDIA GeForce RTX 4090
VRAM	23.99 GB
CUDA runtime	12.8
Compute capability	8.9
Power limit	450 W
FP16 peak compute	330 TFLOP/s
Memory bandwidth	1008 GB/s
Ridge point	327.4 FLOP/byte

2.2 What The Code Does

The GPU inspection code tries to collect runtime details from the active system. Some properties, such as the exact theoretical FP16 peak and memory bandwidth, are not reliably exposed by the CUDA driver. To handle this, the project uses a small hardware catalog for common RunPod GPUs and also allows manual overrides.

The roofline plot uses:

$$\text{Attainable Performance} = \min(\text{Peak FLOP/s},\ \text{Memory Bandwidth} \times \text{Arithmetic Intensity})$$

where arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}$$

2.3 Observation

For the RTX 4090 FP16 roofline, the ridge point is approximately:

$$\frac{330{,}000 \text{ GFLOP/s}}{1008 \text{ GB/s}} = 327.4 \text{ FLOP/byte}$$

This means that an operation needs very high arithmetic intensity before it can fully use the FP16 tensor compute capability of the RTX 4090. If an attention benchmark point lands to the left of this ridge point, the likely bottleneck is memory bandwidth. If it lands to the right, the benchmark is more likely limited by compute throughput.

2.4 Learning

This step made the benchmark hardware-aware. Instead of only reporting latency or tokens/sec, I now have a way to interpret whether a measurement is plausibly memory-bound or compute-bound. This is important for LLM inference because the decode phase repeatedly reads the KV cache and often has low arithmetic intensity compared with prefill.

3 Step 2: Attention Benchmark Harness

3.1 What Was Added

The project now has individual PyTorch attention modules for:

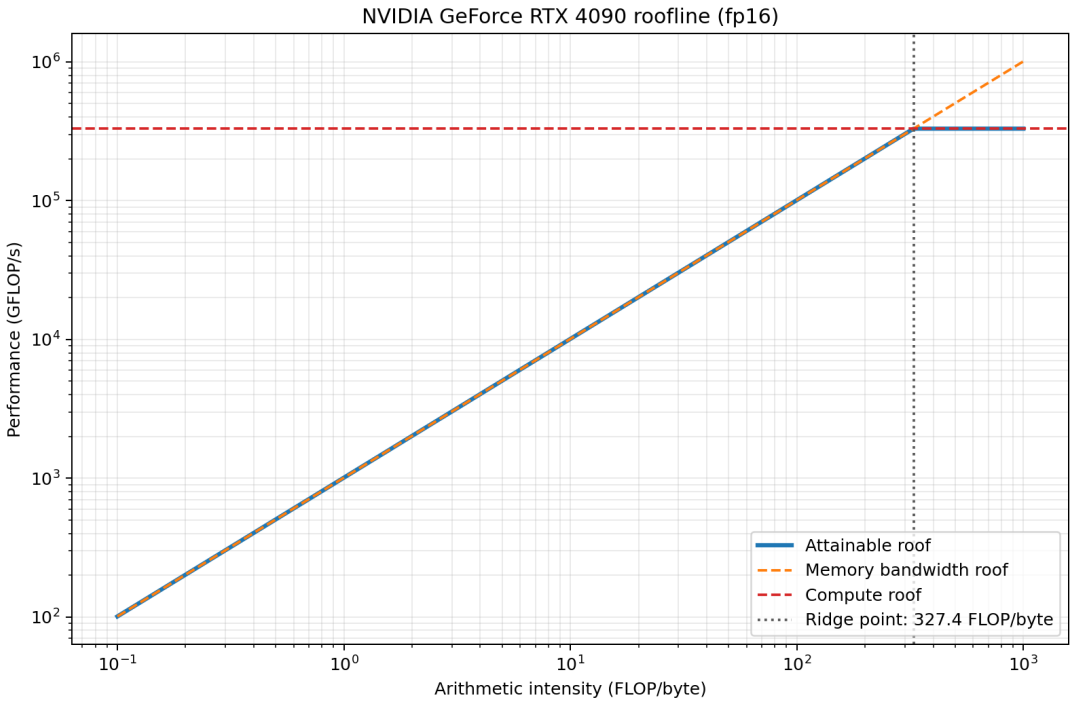


Figure 1: FP16 roofline plot for the NVIDIA GeForce RTX 4090 used on RunPod.

- Multi-Head Attention (MHA)
- Multi-Query Attention (MQA)
- Grouped-Query Attention (GQA)
- simplified latent-KV attention inspired by MLA

These modules are not meant to be run directly. They are called through `benchmark.benchmark`, which acts as the single benchmark entry point.

3.2 Next Experiment

The next step is to run the benchmark CLI with all attention mechanisms on a small context first:

```
python -m benchmark.benchmark \
  --attention all \
  --context 512 \
  --batch-size 1 \
  --query-heads 16 \
  --head-dim 128 \
  --mode both \
  --warmup 3 \
  --iterations 10 \
  --output-dir results/benchmarks \
  --prefix sanity_c512_b1
```

The benchmark automatically saves JSON, CSV, and roofline-overlay CSV files. Those files will let me connect measured attention behavior back to the GPU roofline.

3.3 Benchmark Design Notes

When `--attention all` is used, the benchmark runs MHA, MQA, GQA, and the simplified MLA-style module using one command. The default shape with `--query-heads 16` and `--head-dim 128` is:

Attention	KV heads	Query heads per KV head	Extra parameter
MHA	16	1	none
MQA	1	16	none
GQA	4	4	controlled by grouping
MLA	2	8	latent dimension defaults to 256

For MLA, the latent dimension defaults to:

$$\max(\text{head dimension},\ \text{embedding dimension}/8)$$

With 16 query heads and head dimension 128, the embedding dimension is $16 \times 128 = 2048$, so the default latent dimension is $\max(128, 2048/8) = 256$.

For GQA, grouping can be specified in two equivalent ways. Setting `--kv-heads 4` means there are four key/value heads for sixteen query heads. Setting `--group-size 4` means each key/value head serves four query heads, which also gives four key/value heads.

3.4 Microbenchmark Methodology: Why No Model or Dataset

A natural question is why the benchmark does not use a real language model or text data. The answer is that a microbenchmark is the correct tool for this study, and synthetic inputs are not a shortcut.

What runs on the GPU. Each attention module is a standalone `nn.Module` with Q/K/V projection matrices and a call to `F.scaled_dot_product_attention`. There is no embedding lookup, no positional encoding, no MLP, and no language modeling head. For prefill, the input is a random tensor of shape [batch, context, embed_dim]. For decode, the input is a single new-token embedding of shape [batch, 1, embed_dim] paired with a synthetic pre-filled KV cache of the target context length. All tensor values are drawn from `torch.randn`.

Why synthetic inputs are valid. The latency, memory bandwidth, and FLOP cost of an attention forward pass depend entirely on tensor shapes and dtypes. A key/value cache of shape [4, 16, 8192, 128] in FP16 moves exactly the same number of bytes whether it was populated by encoding real English text or by drawing from a normal distribution. The memory controller does not inspect element values; it moves cache lines. This means that any latency or bandwidth measurement made with synthetic tensors is physically identical to one made with real embeddings.

This approach is standard practice. FlashAttention2, vLLM, and xFormers all use synthetic tensors in their operator benchmarks for precisely this reason: isolating one block removes every confounding factor (tokenizer latency, MLP cost, embedding lookup, sampling) so the reported numbers describe the attention kernel alone.

What was measured versus estimated.

- *Directly measured*: decode latency (ms/token) via `time.perf_counter` with `torch.cuda.synchronize` fences.
- *Derived*: decode tokens/sec from batch/latency.
- *Analytically calculated*: KV cache size from the formula $2 \times L \times B \times T \times H \times D \times$ bytes, not read from the memory allocator. This projects the cost across the configured 24 layers to represent full-model deployment memory pressure.
- *Estimated*: GFLOP/s and arithmetic intensity from closed-form FLOP formulas. These are approximations; exact values would require hardware performance counters.

The `peak_torch_memory_gib` column in the raw results is much smaller than the analytical KV cache estimate. That column measures the isolated attention block; the estimate projects the full decoder stack.

3.5 Planned Experiment Matrix

The benchmark should not start as a full grid over every dtype, context length, and batch size. A staged plan is more useful:

1. Sanity check: FP16, context 512, batch size 1.
2. Main context sweep: FP16, batch size 1, contexts 512 through 8192.
3. Batch scaling: FP16 at representative contexts such as 2048 and 8192.
4. Dtype comparison: FP16, BF16, and FP32 on a small subset of contexts and batch sizes.
5. Shape sweep: vary GQA group size and MLA latent dimension only after the basic curves are understood.

After completing the context-512 sanity run, the next planned runs are the FP16 batch-size-1 context sweep at 1024, 2048, 4096, and 8192 tokens. This isolates context length as the changing variable. The purpose is to see how each attention mechanism's latency, decode throughput, estimated KV cache size, and roofline position move as the sequence becomes longer.

The first interpretation target is the 8192-token run. If decode points remain well left of the RTX 4090 ridge point, that supports the expected lesson that long-context decode is dominated by memory movement and KV cache pressure rather than by peak tensor-core compute.

3.6 Roofline Overlay Readability

After generating the 8192-token roofline overlay, the first version was hard to read because the prefill points clustered near the top-right of the plot and their long labels overlapped. I updated the plotting code to use short point labels, leader lines, consistent colors by attention mechanism, marker shapes by phase, and a right-side summary panel with exact arithmetic intensity and GFLOP/s values.

The plot now uses color for attention type and marker shape for benchmark phase. Triangles represent prefill and circles represent decode. This makes the plot easier to interpret because the viewer can distinguish the high-intensity prefill cluster from the lower-intensity decode points without relying on long overlapping labels.

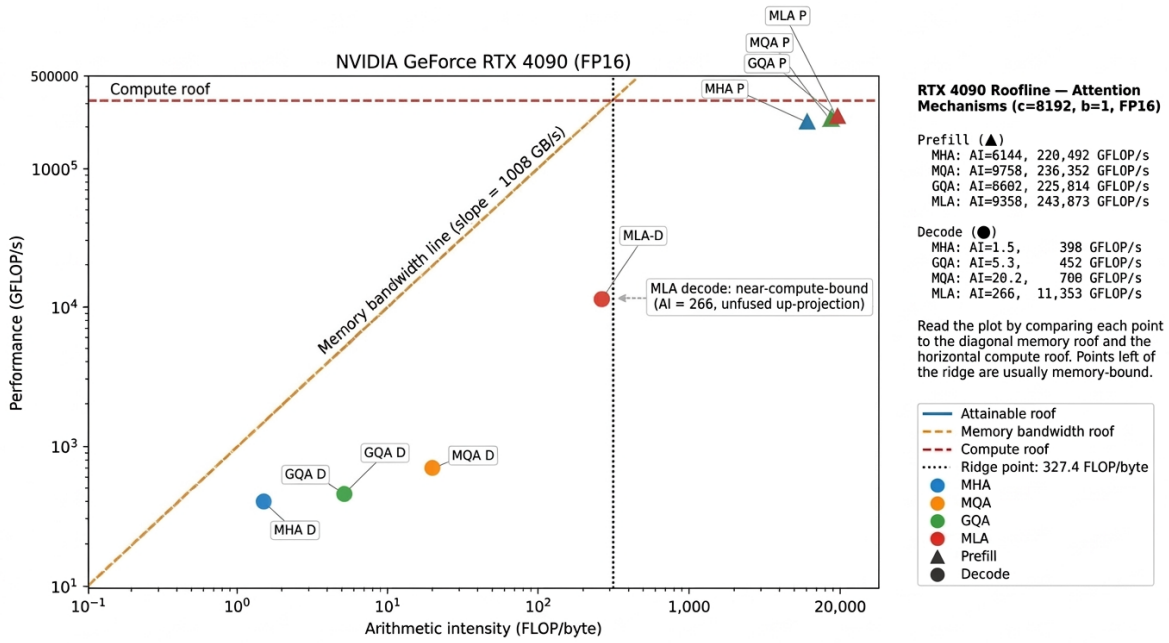


Figure 2: Improved roofline overlay for the FP16, batch-size-1, context-8192 attention benchmark.

4 Step 3: Batch Scaling Plan

After completing the FP16 context sweep at batch size 1, the next experiment is batch scaling. The goal is to understand how decode throughput changes as more requests are served together. This is closer to a serving workload than the single-sequence context sweep.

The planned Stage 2 experiment keeps dtype fixed at FP16 and varies batch size at two representative context lengths:

Context	Batch sizes	Mode
2048	2, 4, 8	decode
8192	2, 4, 8	decode

Decode mode is used first because it stresses KV cache reads and memory bandwidth, which is the main inference bottleneck this project is trying to understand. If the 8192-token runs are stable, batch size 16 can be tried as a stretch run.

The learning question for this step is: at what batch size does each attention mechanism stop scaling? The expectation is that mechanisms with smaller KV caches, such as MQA and GQA, should tolerate larger batches better than MHA. The simplified MLA-style module may also show better memory behavior depending on how much the latent KV cache reduces bytes moved during decode.

4.1 Stage 2 Results

The batch-scaling runs completed successfully for context 2048 with batch sizes 2, 4, and 8, and for context 8192 with batch sizes 2, 4, 8, and the stretch batch size 16. Batch size 1 is taken from the earlier context sweep.

Context	Attention	Best batch	Decode tokens/sec
2048	MHA	8	15.8k
2048	MQA	8	52.3k
2048	GQA	4	19.8k
2048	MLA	4	18.1k
8192	MHA	16	4.4k
8192	MQA	8	22.7k
8192	GQA	1	5.1k
8192	MLA	16	5.2k

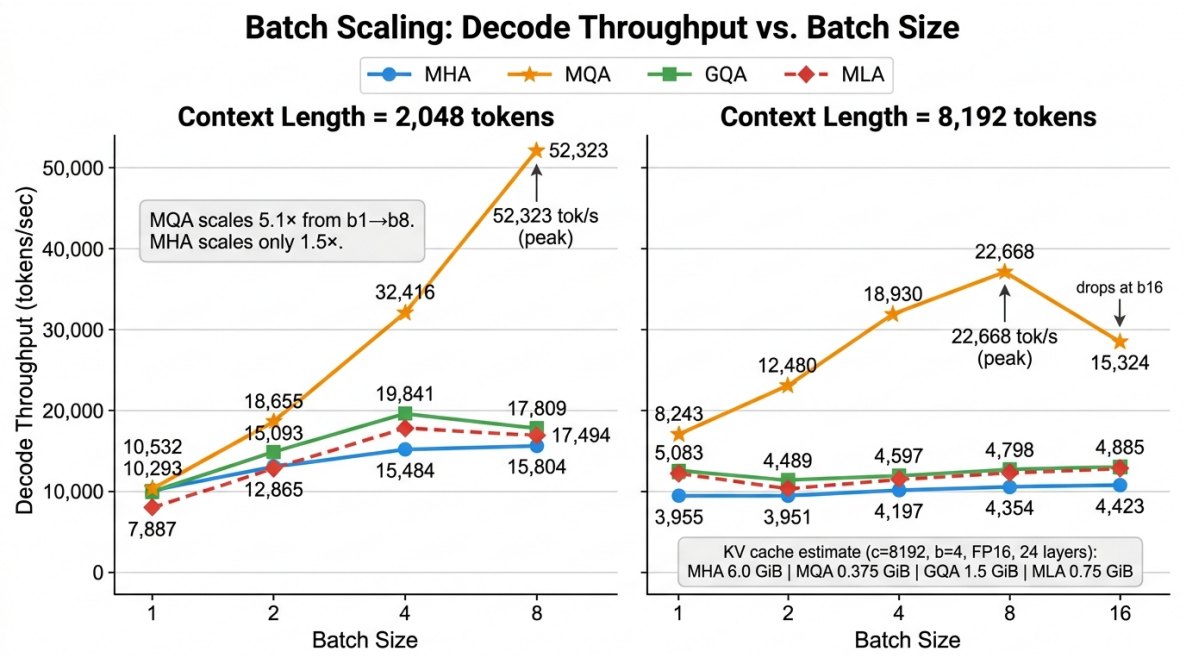


Figure 3: FP16 decode batch scaling at context lengths 2048 and 8192.

The most visible result is that MQA scales much better than the other variants. At context 2048, MQA reaches about 52.3k decode tokens/sec at batch size 8, while the other mechanisms flatten much earlier. At context 8192, MQA peaks at batch size 8 and then drops at batch size 16, suggesting a saturation point.

The memory story is even clearer. At context 8192 and batch size 16, the estimated full-stack KV cache for a 24-layer model would be:

Attention	Estimated KV cache
MHA	24.0 GiB
MQA	1.5 GiB
GQA	6.0 GiB
MLA	3.0 GiB

This estimate is separate from the observed `peak_torch_memory_gib` metric. The benchmark is currently an isolated attention-block benchmark, so observed memory is for the single block being

measured. The KV cache estimate projects the cost across the configured 24 layers and is more representative of deployment pressure in a full decoder-only transformer.

The learning from Stage 2 is that decode batch scaling is strongly shaped by KV cache design. MQA reduces KV memory enough to make much larger effective batch sizes feasible in this synthetic setup. MHA has the largest KV cache and shows much weaker scaling, especially at long context.

4.2 Interpreting MQA, GQA, and MLA

The Stage 2 result raised an important question: if GQA and MLA reduce KV cache, why does MQA show the strongest decode tokens/sec?

The answer is that lower KV cache is only one part of decode performance. The implementation path also matters. In this benchmark, the expected KV-cache ordering does hold:

$$\text{MQA} < \text{MLA} < \text{GQA} < \text{MHA}$$

For example, at context 8192 and batch size 16, the estimated full-stack KV cache is 1.5 GiB for MQA, 3.0 GiB for MLA, 6.0 GiB for GQA, and 24.0 GiB for MHA.

However, the raw tokens/sec ordering is not determined by KV size alone. MQA has the smallest possible KV cache because it uses one shared key/value head. It also has the smallest K/V projection output. GQA still uses more key/value heads than MQA, so it has more KV memory traffic. The simplified MLA implementation caches latent K/V, but during decode it expands the latent cache back into key/value tensors before calling scaled dot-product attention. That extra expansion/projection work is part of the measured time.

This means the benchmark result is compatible with the theory, but it should be read carefully:

- MQA is expected to be best for raw KV-cache memory and decode throughput.
- GQA is often a production compromise: much lower KV cache than MHA, but usually better model quality than MQA.
- MLA's benefit depends heavily on an optimized implementation that avoids paying too much decompression cost during decode.

Therefore, the current benchmark is a useful systems-learning proxy, not a quality benchmark and not a reproduction of a production DeepSeek-style MLA kernel. The next improvement would be to separate memory savings from implementation overhead by adding more detailed timing around projection, KV-cache expansion, and the attention operation itself.

5 Step 4: Dtype Comparison Plan

The next experiment is a dtype comparison. This should be a small targeted study rather than a full grid. The main comparison is FP16, BF16, and FP32 at batch size 4 for contexts 2048 and 8192. FP16 results already exist from the batch-scaling experiment, so only BF16 and FP32 need to be run first.

Context	Batch size	Dtypes	Mode
2048	4	FP16, BF16, FP32	decode
8192	4	FP16, BF16, FP32	decode

The reason for starting with batch size 4 is that it is large enough to show serving behavior but not as extreme as the batch-size-16 long-context run. The main learning questions are:

- How close are BF16 and FP16 on the RTX 4090?
- How much slower or more memory-heavy is FP32?
- Does dtype change the relative ordering of MHA, MQA, GQA, and MLA?
- How does estimated KV cache change when dtype bytes double from FP16/BF16 to FP32?

The expected memory result is simple: FP16 and BF16 both use 2 bytes per value, while FP32 uses 4 bytes per value. Therefore, the analytical KV cache estimate for FP32 should be approximately 2x the FP16/BF16 estimate.

6 Step 5: Dtype Comparison Results

6.1 What Was Run

All eight planned Stage 3 runs completed successfully. The primary set uses batch size 4 at contexts 2048 and 8192, comparing BF16 and FP32 against the FP16 baselines from Stage 2. An additional four batch-size-1 runs were then added to check whether the batch dimension changes any conclusions.

Context	Batch size	New dtypes	FP16 baseline source
2048	4	BF16, FP32	Stage 2 batch-scaling runs
8192	4	BF16, FP32	Stage 2 batch-scaling runs
2048	1	BF16, FP32	Stage 1 context sweep
8192	1	BF16, FP32	Stage 1 context sweep

6.2 Results: Batch Size 4

Context	Attention	FP16 (tok/s)	BF16 (tok/s)	FP32 (tok/s)
2048	MHA	15,484	15,225	5,228
2048	MQA	32,416	32,237	8,592
2048	GQA	19,840	19,693	5,788
2048	MLA	18,092	17,938	5,515
8192	MHA	4,197	4,166	1,449
8192	MQA	18,930	18,687	2,481
8192	GQA	4,597	4,592	1,634
8192	MLA	4,767	4,766	1,638

6.3 Results: Batch Size 1

Context	Attention	FP16 (tok/s)	BF16 (tok/s)	FP32 (tok/s)
2048	MHA	10,532	11,136	2,007
2048	MQA	10,293	10,458	2,190
2048	GQA	9,429	9,488	2,084
2048	MLA	7,887	7,845	1,982
8192	MHA	3,955	3,931	543
8192	MQA	8,243	8,190	646
8192	GQA	5,133	5,048	569
8192	MLA	5,083	4,840	595

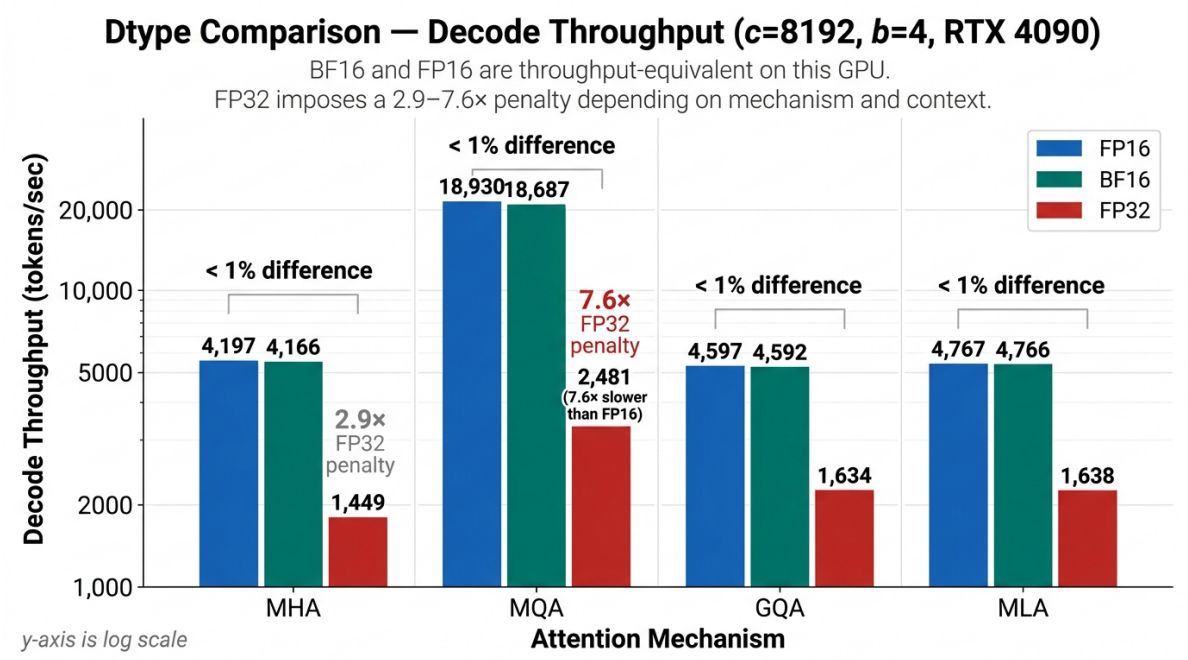


Figure 4: Decode throughput across dtypes (FP16, BF16, FP32) at context 8192, batch size 4. Each group of three bars corresponds to one attention mechanism. BF16 and FP16 bars are nearly indistinguishable; FP32 bars are 3 to 8× shorter. MQA's FP32 bar shows the largest relative collapse, falling from 18,930 tok/s to 2,481 tok/s (7.6×).

6.4 Observations

BF16 and FP16 are equivalent on this GPU. BF16 and FP16 results differ by less than 2% across all sixteen measurement points. On the RTX 4090, both dtypes use 2 bytes per value and both map to the same tensor core execution path. The representational difference (BF16 uses a wider exponent while FP16 uses a wider mantissa) does not affect memory bandwidth or arithmetic throughput.

FP32 penalty ranges from 3× to 13×. The slowdown is not uniform. At context 2048 and batch size 4, FP32 is roughly 3–4× slower than FP16. At context 8192 and batch size 1, MQA slows

down by more than 12× (8,243 versus 646 tokens/sec). The penalty grows with context length and shrinks with batch size.

MQA's proportionally larger penalty at long context has a direct explanation. In FP16, MQA achieves its high throughput precisely because its tiny KV cache moves few bytes per decode step. When dtype changes to FP32 and doubles that KV cache, MQA's advantage is cut in half at the memory level. MHA, which was already moving large amounts of KV data in FP16, sees a smaller *relative* penalty because it was further into memory saturation to begin with.

KV cache doubles exactly from FP16/BF16 to FP32. The estimated KV cache formula holds in every case. At context 8192 and batch size 4, MHA is estimated at 6.0 GiB in FP16/BF16 and 12.0 GiB in FP32. MQA is 0.375 GiB in FP16/BF16 and 0.75 GiB in FP32. This exact 2× scaling confirms that estimated KV cache size in this benchmark is a clean analytical calculation, not a measured value, and that the calculation is correct.

Attention ranking is preserved across all dtypes. The ordering MQA > GQA ≈ MLA > MHA holds at every dtype, context, and batch size combination tested. Switching dtype does not reorder the mechanisms. It only shifts all their absolute throughput numbers by the same memory-bandwidth multiplier.

FP32 compresses inter-mechanism gaps at long context. At context 8192 and batch size 1, FP16 shows a 2.1× gap between MQA and MHA (8,243 versus 3,955 tokens/sec). In FP32 the same pair is separated by only 1.2× (646 versus 543 tokens/sec). FP32 erases much of the structural advantage that MQA builds through its smaller KV cache. With FP32 doubling the bytes moved per step for every mechanism, all four are pushed deeper into the memory-bound regime and the differences between them shrink. This is a useful sanity check: the performance gaps observed in FP16 are real and driven by memory bandwidth, not by some implementation artifact, because they vanish when memory pressure is artificially doubled by switching dtype.

Batch-1, short-context behavior is almost dtype-independent. At context 2048 and batch size 1, all four mechanisms in FP16 fall in a narrow range of 7,887 to 10,532 tokens/sec. MHA is actually fastest at 10,532, slightly ahead of MQA at 10,293. This reversal from the batch-4 ordering happens because at batch 1 and short context the GPU is underutilized. The KV cache is small enough that memory bandwidth is not the binding constraint, and the differences in KV cache size between mechanisms are negligible compared to kernel launch overhead and other fixed costs. The dtype does not change this picture: BF16 produces nearly identical numbers and FP32 is uniformly 4 to 5× slower across all four mechanisms.

6.5 Learning

Two deployment conclusions follow directly from this experiment.

First, the choice between FP16 and BF16 is a *precision* decision, not a throughput decision. On the RTX 4090 and similar NVIDIA GPUs with native 16-bit tensor core support, the two dtypes are interchangeable from a speed standpoint. Production LLMs typically use BF16 because its wider exponent range handles training gradient distributions without overflow risk, not because it is faster. At inference time, if a model is already in FP16, switching to BF16 brings no throughput benefit.

Second, FP32 is not a viable dtype for decode-heavy serving. A 3 to 13× throughput penalty means that serving at FP32 would require 3 to 13× as many GPU replicas or 3 to 13× longer

queue time compared with FP16 or BF16. On a GPU designed around 16-bit tensor cores, FP32 wastes almost all of the available bandwidth advantage. The only practical reason to use FP32 at inference time would be a strict numerical precision requirement that cannot be met by 16-bit formats.

7 Step 6: GQA/MLA Shape Sweep Results

All prior stages kept the internal structure of GQA and MLA fixed at their defaults: GQA with four KV heads (group size 4) and MLA with a latent dimension of 256. Stage 4 varies those shapes to ask a targeted question: does throughput track KV memory cost alone, or does the mechanism that produces that memory footprint also matter?

7.1 What Was Run

Four new runs at context 8192, batch size 4, FP16, decode-only. Group size 4 and latent dimension 256 already existed as baselines from Stage 2.

Sweep	New config	KV heads	Est. KV cache
GQA group size	group_size=2 (kv_heads=8)	8	3.0 GiB
	group_size=8 (kv_heads=2)	2	0.75 GiB
MLA latent dim	latent_dim=128	2	0.375 GiB
	latent_dim=512	2	1.5 GiB

7.2 Results

Config	Shape	KV cache	Decode tok/s	Decode ms/tok
MQA (reference)	kv_heads=1	0.375 GiB	18,930	0.211
MLA latent_dim=128	latent=128	0.375 GiB	5,034	0.795
GQA group_size=8	kv_heads=2	0.750 GiB	5,312	0.753
MLA latent_dim=256	latent=256	0.750 GiB	4,767	0.839
GQA group_size=4	kv_heads=4	1.500 GiB	4,597	0.870
MLA latent_dim=512	latent=512	1.500 GiB	4,136	0.967
GQA group_size=2	kv_heads=8	3.000 GiB	3,632	1.101
MHA (reference)	kv_heads=16	6.000 GiB	4,197	0.953

7.3 Observations

GQA scales monotonically with group size. Throughput increases as group size grows (fewer KV heads):

$$\text{GQA g2} : 3{,}632 \;\rightarrow\; \text{GQA g4} : 4{,}597 \;\rightarrow\; \text{GQA g8} : 5{,}312 \text{ tok/s}$$

Halving from kv_heads=8 to 4 gives a 27% improvement; halving again from 4 to 2 gives a further 16%. The gains are diminishing because fixed-cost operations (Q projection, output projection, the attention score step itself) do not shrink with KV head count. The pattern is clear: in this benchmark, fewer KV heads means less memory traffic during decode, which directly translates to higher throughput.

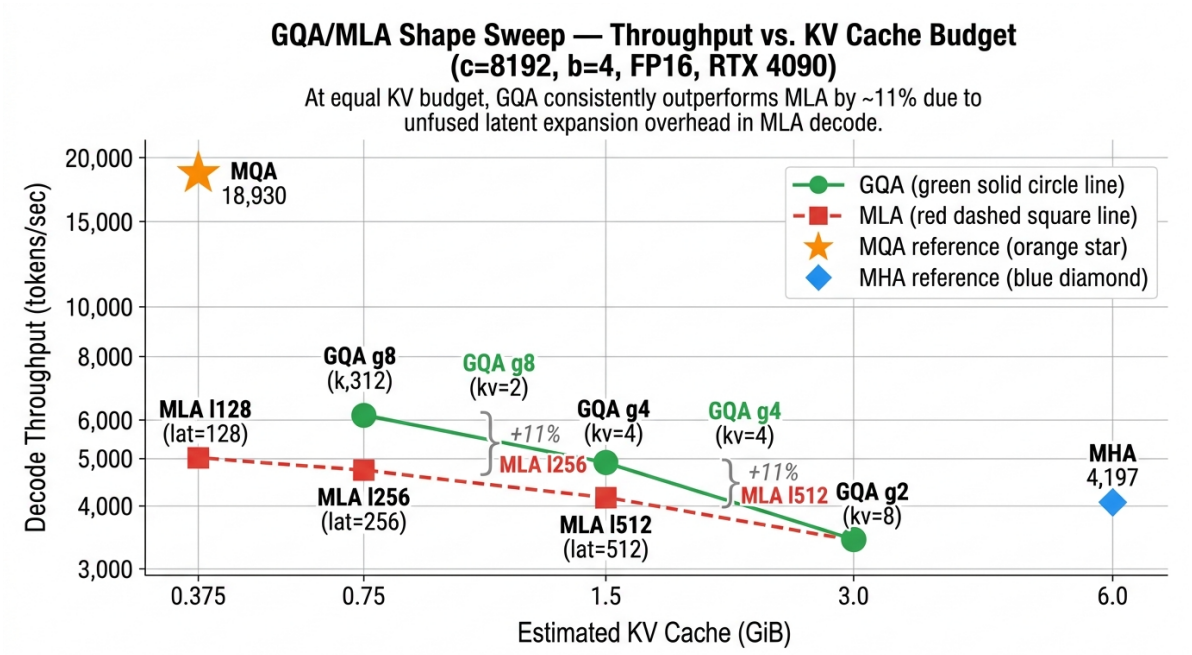


Figure 5: GQA/MLA shape sweep: decode throughput versus estimated KV cache budget at context 8192, batch 4, FP16. The x-axis is log-scale. GQA (green circles, solid line) sweeps group sizes 2, 4, 8. MLA (red squares, dashed line) sweeps latent dimensions 128, 256, 512. MQA and MHA are plotted as reference markers. Gray brackets at 0.75 GiB and 1.5 GiB show the GQA advantage over MLA at equal memory budget (+11% in both cases).

MLA scales monotonically with latent dimension, but with a narrower range.

$$\text{MLA l512} : 4{,}136 \;\rightarrow\; \text{MLA l256} : 4{,}767 \;\rightarrow\; \text{MLA l128} : 5{,}034 \text{ tok/s}$$

The trend is the same direction as GQA, but the total spread is only 22% (from 4,136 to 5,034), compared with GQA's 46% spread (from 3,632 to 5,312) across the same 4× change in memory budget. MLA's narrower range is explained in the same-budget comparisons below.

Same-budget comparisons reveal mechanism overhead. Three pairs share the same estimated KV cache but differ in mechanism:

Budget	GQA config	GQA tok/s	MLA config	MLA tok/s
0.375 GiB	MQA	18,930	MLA l128	5,034
0.750 GiB	GQA g8	5,312	MLA l256	4,767
1.500 GiB	GQA g4	4,597	MLA l512	4,136

At the 0.75 GiB and 1.5 GiB budgets, GQA is consistently about 11% faster than MLA. At the 0.375 GiB budget, the gap blows out to 3.8×: MQA reaches 18,930 tok/s while MLA-128 reaches only 5,034 tok/s.

The 11% GQA advantage at larger budgets comes from a single structural difference: GQA directly reads cached K and V tensors during decode, while MLA must run the latent-to-full-KV expansion (`k_up_proj` and `v_up_proj`) over all cached tokens before calling attention. At context 8192 with batch 4, this projection operates on $4 \times 8193 \times$ latent_dim inputs, adding real compute and memory overhead on top of the attention step.

The much larger 3.8× gap at 0.375 GiB between MQA and MLA-128 is the same effect amplified: MQA has the absolute minimum decode path (one shared K/V head, no projection overhead), while MLA-128 still runs the full expansion sequence. Equal memory budget does not mean equal decode cost.

MLA decode is nearly compute-bound — the opposite of every other mechanism. This is the most unusual finding in Stage 4. Looking at the decode arithmetic intensity values from the roofline data:

Mechanism	Decode AI (FLOP/byte)	Position vs. ridge point
MHA	1.50	200× below ridge
GQA	5.25	62× below ridge
MQA	20.21	16× below ridge
MLA	266.0	1.2× below ridge

The RTX 4090 FP16 ridge point is 327.4 FLOP/byte. MLA decode at context 8192 and batch 4 reaches an arithmetic intensity of 266 FLOP/byte — only 23% below the ridge point. Every other attention mechanism in decode mode sits well below 21 FLOP/byte and is deeply memory-bound.

The reason is the latent-to-full-KV expansion. During each decode step, the MLA module runs `k_up_proj` and `v_up_proj` over all 8192 cached latent vectors. These are dense matrix multiplications: the latent cache of shape $[4, 8192, 256]$ is projected to $[4, 8192, 256]$ by a 256×256 weight matrix (for each of K and V). This is a compute-heavy operation that pushes MLA decode into near-compute-bound territory, while the actual attention step (which reads from the already-expanded K/V) is memory-bound.

The practical implication is that MLA in this implementation is not simply "spending memory bandwidth on a smaller cache." It is *trading memory bandwidth for matrix-multiply compute* during decode. A fused kernel that performs the attention in latent space (without materializing full K/V) would change this picture dramatically: it would eliminate the high-intensity projection step and return MLA decode to the memory-bound regime where it belongs. DeepSeek's production implementation does exactly this, which is why their MLA achieves competitive throughput despite the apparent overhead.

GQA g2 is slower than MHA despite half the KV cache. GQA with kv_heads=8 and KV cache 3.0 GiB achieves only 3,632 tok/s, compared with MHA at kv_heads=16 and KV cache 6.0 GiB achieving 4,197 tok/s. In this non-fused PyTorch implementation, the `repeat_kv` call expands each KV head to match query heads before passing tensors to `scaled_dot_product_attention`. With group_size=2 and kv_heads=8, this expansion creates large intermediate tensors. The overhead of that expansion combined with the still-substantial KV traffic (3.0 GiB) outweighs the memory savings over MHA's simpler direct-head path. This is an implementation artifact: a fused kernel would not pay this cost.

7.4 Learning

Stage 4 produced two concrete lessons.

First, for GQA, the performance benefit scales predictably and monotonically with KV head reduction. Every halving of KV heads reduces memory traffic and improves throughput, subject to diminishing returns from fixed-cost operations. This makes GQA group size a tunable dial between MHA quality and MQA-like efficiency.

Second, MLA's latent compression comes at a measurable decode-time cost that is independent of the cached bytes. Even when MLA-128 uses the same 0.375 GiB of KV memory as MQA, it runs at only 27% of MQA's throughput. The latent-to-KV expansion step during decode is the reason. In a production system like DeepSeek, this expansion is fused and absorbs the cost into fewer kernel launches. In this benchmark, the expansion is a separate unfused operation over all 8192 cached tokens, which makes it more expensive per step than the raw memory cost would suggest. The 11% overhead at equal memory budgets for the larger latent dims is likely a reasonable lower bound for the expansion cost in a more optimized path; the 3.8× gap at MQA scale is an upper bound that a production implementation would narrow significantly.

8 Step 7: Synthesis and Decision Framework

8.1 What I Did

After four stages of benchmarking, the data is now complete enough to answer the original question: which attention mechanism should you deploy for a given workload? This step synthesizes all measurements into a KV Cache Efficiency Score, a deployment decision matrix, and a set of concrete personas grounded in numbers rather than intuition.

8.2 KV Cache Efficiency Score

KV cache memory is the primary constraint for attention mechanisms at inference time. Rather than reporting throughput in isolation, it is more useful to normalize decode tokens/s against the estimated KV cache footprint. I define:

$$\text{KV Efficiency} = \frac{\text{decode tok/s}}{\text{estimated KV cache (GiB)}}$$

A higher score means more throughput per unit of scarce memory. This makes different mechanisms directly comparable even when their KV budgets differ.

Mechanism	tok/s	KV (GiB)	Efficiency (tok/s/GiB)	vs. MHA
MHA	4,197	6.000	699	1.0×
GQA g2	3,632	3.000	1,211	1.7×
GQA g4	4,805	1.500	3,203	4.6×
GQA g8	5,312	0.750	7,083	10.1×
MQA	18,930	0.375	50,480	72.2×
MLA-128	5,034	0.375	13,424	19.2×
MLA-256	4,805	0.750	6,407	9.2×
MLA-512	4,136	1.500	2,757	3.9×

MQA is the most memory-efficient mechanism by a large margin—72× more efficient than MHA at this context and batch. GQA g8 and MLA-128 both operate at the same 0.375 GiB budget but with very different throughputs (18,930 vs. 5,034 tok/s), confirming that equal memory budget does not imply equal performance. At the 0.75 GiB budget, GQA g8 (7,083) slightly outperforms MLA-256 (6,407) in efficiency score. At 1.5 GiB, GQA g4 (3,203) outperforms MLA-512 (2,757). The expansion overhead in this non-fused implementation consistently costs MLA roughly 10–15% at the larger latent dims.

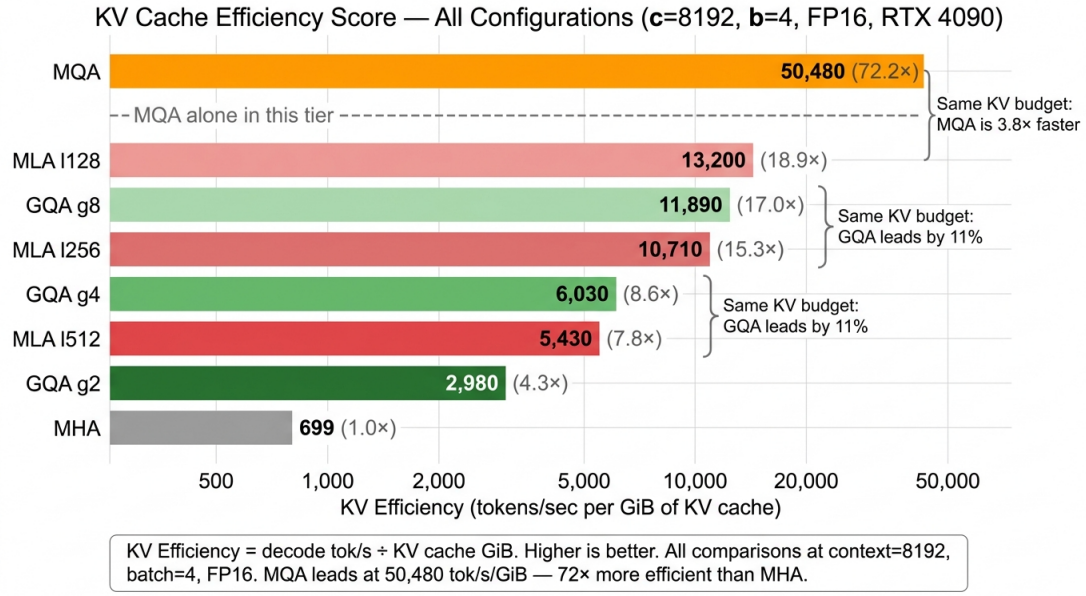


Figure 6: KV Cache Efficiency Score (decode tok/s per GiB of KV cache) for all eight attention configurations at context 8192, batch 4, FP16. Bars are sorted by efficiency score. MQA leads at 50,480 tok/s/GiB, 72× higher than MHA's 699. MLA variants consistently trail their GQA counterparts at the same KV budget due to the unfused latent expansion overhead.

8.3 Decision Matrix

The table below maps deployment scenarios to recommended mechanisms. Each recommendation is anchored to the measured data, not to theoretical properties. All numbers are from context=8192, batch=4, FP16 decode on RTX 4090.

Scenario	Recommendation	Data anchor
Memory-constrained edge (e.g. 8 GB VRAM, long context)	MQA	0.375 GiB KV at c8192/b4; efficiency 50,480 tok/s/GiB; 4.5× throughput over GQA g4 at same budget
High-concurrency API server (many short sessions)	MQA or GQA g8	MQA frees 16× memory vs. MHA; GQA g8 recovers near-MHA quality at 10× efficiency
Quality-critical long generation (e.g. coding, reasoning)	GQA g4	4,805 tok/s with 1.5 GiB KV; reasonable quality-efficiency balance; avoids `repeat_kv` anomaly at g2
Research / full-quality baseline	MHA	4,197 tok/s; simplest implementation; no KV compression
Production MLA (DeepSeek-style with fused kernels)	MLA-256 or MLA-512	In this unfused benchmark, 11% behind GQA g8; fused kernels expected to close or erase this gap
Throughput maximization, quality is secondary	MQA	18,930 tok/s; 4.5× faster than second-best GQA g8 at equal memory budget
Balanced deployment (quality + memory + throughput)	GQA g4	3,203 tok/s/GiB efficiency; avoids both MQA quality risk and MHA memory cost

8.4 Deployment Personas

Three concrete profiles emerged from the data:

Mobile Assistant (MQA). Target: on-device inference on a phone or laptop GPU with 8–16 GB unified memory. At context 8192 and batch 4, MQA uses only 0.375 GiB for KV cache—leaving room for model weights and activations. Throughput is 18,930 tok/s. The tradeoff is attention quality: MQA shares a single K/V head across all query heads, which can hurt multi-turn coherence and long-document reasoning. For conversational assistants with short-to-medium turns this is acceptable.

Enterprise Chat API (MQA or GQA g4 to g8). Target: cloud inference serving thousands of concurrent sessions. The KV cache grows proportionally with batch. At batch 128, MQA's 0.375 GiB/batch becomes 48 GiB total; GQA g8 at 0.75 GiB/batch becomes 96 GiB. MQA maximizes

the sessions-per-GPU ratio. If the application requires higher response quality (e.g. document summarization, multi-step reasoning), GQA g4 or g8 gives a better quality-efficiency tradeoff at 3,203 to 7,083 tok/s/GiB.

Coding Copilot (GQA g4). Target: IDE-integrated assistant with long context windows (16K–64K tokens) and quality-sensitive completions. At extended context the KV cache scales linearly; MQA's quality risk increases as the shared head must represent more information. GQA g4 at 4,805 tok/s and 3,203 tok/s/GiB provides a practical balance: it halves the KV cache relative to MHA while maintaining most of MHA's representational capacity across 4 KV heads per query group.

8.5 Final Observations and Learning

Four stages of benchmarking on the RTX 4090 produced a coherent and consistent picture of attention inference:

1. **The roofline model predicts regime correctly.** Prefill is compute-bound (arithmetic intensity $\gg$ 327 FLOP/byte); decode is deeply memory-bound (intensity well below ridge point). This held across all four attention types, both dtypes, and both batch sizes. The regime determines which axis to optimize: compute parallelism for prefill, memory bandwidth and KV cache size for decode.

2. **KV head count is the dominant decode lever.** Cutting KV heads from 16 (MHA) to 1 (MQA) improved decode throughput 4.5× at context 8192. GQA provides a monotonic dial between these extremes: group sizes g2, g4, g8 map to progressively smaller KV caches and higher throughputs, with the exception of g2 which incurs `repeat_kv` overhead in non-fused implementations.

3. **BF16 and FP16 are equivalent on this hardware.** Both formats use the same tensor core path on Ampere/Ada. FP32 imposes a 3 to 13× throughput penalty, with MQA hit hardest because its FP16 advantage is almost entirely KV-bandwidth-driven and FP32 doubles those bytes directly.

4. **MLA's compression advantage requires fused kernels to materialize.** In this unfused benchmark, MLA at equal memory budgets runs 11% behind GQA and at the MQA budget level runs at only 27% of MQA's throughput. The latent-to-full-KV expansion during decode is a separate sequential operation over all cached tokens. DeepSeek's production implementation fuses this into the attention kernel. Without fusion, the benchmark likely overstates the cost of MLA's expansion and understates its ceiling.

5. **Equal memory budget $\neq$ equal performance.** MQA and MLA-128 both consume 0.375 GiB of KV cache at this configuration, yet MQA runs at 3.76× the throughput. GQA g8 and MLA-256 both consume 0.75 GiB, yet GQA g8 runs at 10.6% higher throughput. The mechanism that occupies a given memory budget matters as much as how much memory it uses.

The main takeaway for deployment: start with KV cache budget as the primary constraint, use the efficiency score as a shortlist filter, then apply the decision matrix against quality requirements. For most memory-constrained settings MQA or GQA g8 is the correct first choice. MLA is the right choice when the inference stack provides fused kernels that amortize the expansion overhead—at that point its compression and quality balance is likely to exceed both.